综合练习

人员

许睿谦、阮文璋 到课

上周作业检查

上周作业链接: https://vjudge.net/contest/821864

Rank	Team	Score	Penalty	A 12 / 16	B 12 / 28	C 12 / 15	D 12 / 28	E 4 / 14
1	☆ B_neutrino (Vm1m3)	5	2023	5:15:56 (-1)	5:27:15	5:44:52 (-1)	6:18:07	**9:57:07 (-1)**
2	☆ xuruiqian	5	7807	5:15:23	5:34:00 (-1)	5:53:00 (-1)	6:25:39 (-4)	4:08:59:05
3	☆ wcz123bc	5	9126	0:24:35	0:33:08	0:47:49	1:18:34	6:03:42:31 (-4)
4	☆ lyc12345bc (刘奕辰)	5	9786	0:20:07	0:38:00 (-1)	0:48:58	1:20:27 (-2)	6:13:59:20 (-3)
5	☆ chenxinmiao (陈欣妙)	4	198	0:14:59	0:36:00 (-1)	0:48:00	1:19:03	
6	☆ yyycj	4	210	0:17:28	**0:25:14**	**0:44:40 (-1)**	1:23:30 (-1)	
7	☆ chujinxuan (褚锦轩)	4	229	0:14:58	0:30:00 (-2)	0:50:00	**1:14:35 (-1)**	
8	☆ junyan007008	4	254	0:23:58 (-2)	0:38:47	0:52:42	1:18:46 (-1)	(-2)
9	☆ ccs2012 (陈丛晟)	4	296	**0:14:28**	0:37:00 (-1)	0:54:00	1:50:54 (-3)	
10	☆ 7654Fox (隋天乙)	4	317	**0:14:28 (-1)**	0:53:00 (-2)	1:02:00	1:47:46 (-1)	
11	☆ lijinshu	4	9703	0:18:24	1:04:00 (-8)	6:11:41:00	1:39:37 (-1)	
12	☆ r111	4	24153	2:13:22:08	3:12:19:00	3:13:28:00	7:02:44:15 (-2)	

本周作业

https://vjudge.net/contest/823546 (课上讲了上上周的 A ~ E 题, 课后作业是本周的 A ~ E 题)

课堂表现

今天的题目难度会相对低一些, 要求同学们课下尽量把 5 道题目都补完。

课堂内容

CF9D How many trees?

维护 f[i][j]: i 个点, 高度 <=j, 有多少方案

答案: f[n][n] - f[n][H-1]

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 35 + 5;
int f[maxn][maxn];  // f[i][j]: i 个点, 高度 <=j, 有多少方案

signed main()
{
  int n, H; cin >> n >> H;
  for (int i = 0; i <= n; ++i) f[0][i] = 1;

  for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= n; ++j) {
      for (int k = 0; k <= i-1; k++) f[i][j] += f[k][j-1]*f[i-1-k][j-1];
    }
  }

  cout << f[n][n] - f[n][H-1] << endl;
  return 0;
}
```

CF1272E Nearest Opposite Parity

建反图, 在反图上以 奇数点 和 偶数点 为起点跑两次 bfs

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2e5 + 5;
int w[maxn];
vector<int> vec[maxn];
int dis[maxn], ans[maxn];

void bfs(int _st) {
  memset(dis, -1, sizeof(dis));
  queue<int> q; q.push(_st); dis[_st] = 0;
  while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int i : vec[u]) {
      if (dis[i] == -1) q.push(i), dis[i] = dis[u]+1;
    }
```

```
    }
  }

  signed main()
  {
    int n; cin >> n;
    for (int i = 1; i <= n; ++i) {
      cin >> w[i];
      if (i-w[i] >= 1) vec[i-w[i]].push_back(i);
      if (i+w[i] <= n) vec[i+w[i]].push_back(i);

      if (w[i]&1) vec[n+1].push_back(i);
      else vec[n+2].push_back(i);
    }

    bfs(n+1);
    for (int i = 1; i <= n; ++i) {
      if (w[i]%2 == 0) ans[i] = (dis[i]==-1 ? -1 : dis[i]-1);
    }
    bfs(n+2);
    for (int i = 1; i <= n; ++i) {
      if (w[i]%2 == 1) ans[i] = (dis[i]==-1 ? -1 : dis[i]-1);
    }

    for (int i = 1; i <= n; ++i) cout << ans[i] << " ";
    cout << endl;
    return 0;
  }
```

CF1846G Rudolf and CodeVid-23

先状压, 然后把这些转化关系用图存起来, 最后跑 dijkstra

```
  #include <bits/stdc++.h>
  #define int long long

  using namespace std;

  const int maxn = 1024 + 5;
  const int inf = 0x3f3f3f3f3f3f3f3f;
  struct edge {
    int to, value;
  };
  vector<edge> vec[maxn];

  int strRead() {
    string s; cin >> s;
    reverse(s.begin(), s.end());
    int res = 0;
    for (char i : s) res = res*2 + (i-'0');
    return res;
```

```
}

struct node {
  int id, d;
  bool operator < (const node& p) const { return d < p.d; }
  bool operator > (const node& p) const { return d > p.d; }
};
int dis[maxn];
bool vis[maxn];

void dijkstra(int _st) {
  memset(dis, 0x3f, sizeof(dis));
  priority_queue<node, vector<node>, greater<node>> q;
  memset(vis, false, sizeof(vis));
  q.push({_st, 0}); dis[_st] = 0;
  while (!q.empty()) {
    node u = q.top(); q.pop();
    int id = u.id, d = u.d;
    if (vis[id]) continue;

    for (edge it : vec[id]) {
      if (d+it.value < dis[it.to]) {
        dis[it.to] = d+it.value; q.push({it.to, dis[it.to]});
      }
    }
  }
}

void solve() {
  int n, m; cin >> n >> m;
  for (int i = 0; i < (1<<n); ++i) vec[i].clear();

  int _st = strRead();
  while (m -- ) {
    int value, a, b; cin >> value;
    a = strRead(), b = strRead();
    for (int i = 0; i < (1<<n); ++i) {
      int t = ((i&(~a))|b);
      vec[i].push_back({t,value});
    }
  }

  dijkstra(_st);
  cout << (dis[0]==inf ? -1 : dis[0]) << endl;
}

signed main()
{
  int T; cin >> T;
  while (T -- ) solve();
  return 0;
}
```

CF335B Palindrome

当长度 >=2600 时, 一定有某个字母出现了 >=100 次, 直接输出即可

当长度 <2600 时, 可以先 区间dp 求最长回文子序列, 然后 dfs 输出

```
#include <bits/stdc++.h>

using namespace std;

const int maxn = 2600 + 5;
int f[maxn][maxn];
string s;

void dfs(int l, int r, int cnt) {
  if (l > r) return;
  if (cnt == 0) return;
  if (cnt == 1) { cout << s[l]; return; }

  if (s[l] == s[r]) {
    cout << s[l];
    dfs(l+1, r-1, cnt-2);
    cout << s[r];
    return;
  }

  if (f[l][r] == f[l+1][r]) dfs(l+1, r, cnt);
  else dfs(l, r-1, cnt);
}

int main()
{
  cin >> s;
  int n = s.size();
  s = " " + s;

  if (n >= 2600) {
    map<char, int> mp;
    for (int i = 1; i <= n; ++i) mp[s[i]]++;
    for (auto it : mp) {
      if (it.second >= 100) {
        for (int i = 1; i <= 100; ++i) cout << it.first;
        return 0;
      }
    }
  }

  for (int i = 1; i <= n; ++i) f[i][i] = 1;
  for (int len = 2; len <= n; ++len) {
    for (int i = 1; i+len-1 <= n; ++i) {
      int j = i+len-1;
      if (s[i] == s[j]) f[i][j] = f[i+1][j-1]+2;
      else f[i][j] = max(f[i+1][j], f[i][j-1]);
```

```
    }
  }

  dfs(1, n, min(f[1][n],100));
  return 0;
}
```

AT_abc201_e [ABC201E] Xor Distances

先做一遍 dfs, 求出 根 到 每个点的路径异或值

然后针对每个二进制位进行考虑即可

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int maxn = 2e5 + 5;
const int mod = 1e9 + 7;
struct node {
  int to, value;
};
vector<node> vec[maxn];
int f[maxn];

void dfs(int u, int fa, int c) {
  f[u] = c;
  for (node it : vec[u]) {
    if (it.to == fa) continue;
    dfs(it.to, u, c^it.value);
  }
}

signed main()
{
  int n; cin >> n;
  for (int i = 1; i <= n-1; ++i) {
    int a, b, c; cin >> a >> b >> c;
    vec[a].push_back({b,c}), vec[b].push_back({a,c});
  }

  dfs(1, -1, 0);

  int res = 0;
  for (int i = 0; i <= 60; ++i) {
    int cnt0 = 0, cnt1 = 0;
    for (int j = 1; j <= n; ++j) {
      int t = (f[j]>>i)&1;
      if (t == 0) ++cnt0;
      else ++cnt1;
```

```
        }
        res += ((cnt0*cnt1)%mod) * ((1LL<<i)%mod);
        res %= mod;
    }
    cout << res << endl;
    return 0;
}
```